

Phase 1 Test Specifications

POC-07 — Inventory Management & Procurement System

Total Test Cases: 20 | **Pass Threshold:** 14 of 20 (70%)

UNIT TESTS (8 cases)

TC-07-P1-UNIT-01: SKU Format Generation

```
def test_sku_format():
    from unittest.mock import MagicMock
    from app.services.inventory_service import generate_sku
    mock_db = MagicMock()
    mock_db.query.return_value.filter.return_value.count.return_value = 41
    sku = generate_sku("grocery", mock_db)
    assert sku == "SKU-GRO-0042"
    assert sku.startswith("SKU-GRO-")
```

TC-07-P1-UNIT-02: SKU Different Categories

```
def test_sku_categories():
    from unittest.mock import MagicMock
    from app.services.inventory_service import generate_sku
    mock_db = MagicMock()
    mock_db.query.return_value.filter.return_value.count.return_value = 0
    assert generate_sku("electronics", mock_db).startswith("SKU-ELC-")
    assert generate_sku("clothing", mock_db).startswith("SKU-CLO-")
    assert generate_sku("household", mock_db).startswith("SKU-HHD-")
```

TC-07-P1-UNIT-03: PO Number Format

```
def test_po_number():
    from unittest.mock import MagicMock
    from datetime import date
    from app.services.inventory_service import generate_po_number
    mock_db = MagicMock()
    mock_db.query.return_value.filter.return_value.count.return_value = 41
    po_num = generate_po_number(mock_db)
    year = date.today().year
    assert po_num == f"PO-{year}-0042"
```

TC-07-P1-UNIT-04: Low Stock Alert Triggered

```
def test_low_stock_alert():
    from unittest.mock import MagicMock
    from app.services.inventory_service import check_stock_alerts
    mock_product = MagicMock(); mock_product.id = 1; mock_product.sku = "SKU-GRO-0001"
    mock_product.reorder_point = 20
    mock_stock = MagicMock(); mock_stock.quantity_available = 15 # below reorder point
    mock_db = MagicMock()
    check_stock_alerts(mock_product, mock_stock, mock_db)
    mock_db.add.assert_called_once()
    call_args = mock_db.add.call_args[0][0]
    assert call_args.alert_type == "low_stock"
```

TC-07-P1-UNIT-05: Out of Stock Alert Critical

```
def test_out_of_stock_alert():
    from unittest.mock import MagicMock
    from app.services.inventory_service import check_stock_alerts
    mock_product = MagicMock(); mock_product.id = 1; mock_product.sku = "SKU-GRO-0002"
    mock_product.reorder_point = 20
    mock_stock = MagicMock(); mock_stock.quantity_available = 0
    mock_db = MagicMock()
    check_stock_alerts(mock_product, mock_stock, mock_db)
    call_args = mock_db.add.call_args[0][0]
    assert call_args.alert_type == "out_of_stock"
```

TC-07-P1-UNIT-06: No Alert Above Reorder Point

```
def test_no_alert_above_reorder():
    from unittest.mock import MagicMock
    from app.services.inventory_service import check_stock_alerts
    mock_product = MagicMock(); mock_product.id = 1; mock_product.sku = "SKU-ELC-0001"
    mock_product.reorder_point = 10
    mock_stock = MagicMock(); mock_stock.quantity_available = 50
    mock_db = MagicMock()
    check_stock_alerts(mock_product, mock_stock, mock_db)
    mock_db.add.assert_not_called()
```

TC-07-P1-UNIT-07: Stock Value Calculation

```
def test_stock_value():
    # quantity_on_hand * cost_price summed across all products
```

```
products_data = [  
    {"quantity_on_hand": 100, "cost_price": 50.0}, # 5000  
    {"quantity_on_hand": 50, "cost_price": 200.0}, # 10000  
]  
total = sum(p["quantity_on_hand"] * p["cost_price"] for p in products_data)  
assert total == 15000.0
```

TC-07-P1-UNIT-08: Quantity Available = OnHand - Reserved

```
def test_quantity_available():  
    from app.models import StockLevel  
    stock = StockLevel()  
    stock.quantity_on_hand = 100  
    stock.quantity_reserved = 30  
    assert stock.quantity_available == 70
```

API INTEGRATION TESTS (8 cases)

TC-07-P1-API-01: Create Product Returns 201 with SKU

```
def test_create_product(client, auth_headers, seeded_supplier):  
    payload = {  
        "name": "Basmati Rice 5kg", "category": "grocery",  
        "unit_price": 350.0, "cost_price": 280.0, "unit_of_measure": "box",  
        "reorder_point": 20, "reorder_quantity": 100, "supplier_id":  
        seeded_supplier["id"]  
    }  
    response = client.post("/api/v1/products", json=payload, headers=auth_headers)  
    assert response.status_code == 201  
    data = response.json()  
    assert data["sku"].startswith("SKU-GRO-")
```

TC-07-P1-API-02: Stock Update Creates Movement and Alert

```
def test_stock_update_alert(client, auth_headers, seeded_product):  
    # First set stock below reorder point  
    payload = {"movement_type": "sale", "quantity": -1000,  
               "reference_number": "SALE-001", "notes": "Bulk sale"}  
    response = client.patch(f"/api/v1/products/{seeded_product['id']}/stock",  
                           json=payload, headers=auth_headers)  
    assert response.status_code == 200  
    # Check low stock alert  
    alerts = client.get("/api/v1/stock/low-alerts", headers=auth_headers).json()  
    assert any(p["id"] == seeded_product["id"] for p in alerts)
```

TC-07-P1-API-03: Create Purchase Order with PO Number

```
def test_create_po(client, auth_headers, seeded_supplier, seeded_product):
    payload = {
        "supplier_id": seeded_supplier["id"],
        "order_date": "2026-06-18",
        "expected_delivery": "2026-06-25",
        "items": [{"product_id": seeded_product["id"], "quantity_ordered": 100,
"unit_cost": 280.0}]
    }
    response = client.post("/api/v1/orders", json=payload, headers=auth_headers)
    assert response.status_code == 201
    data = response.json()
    assert "po_number" in data
    assert data["po_number"].startswith("PO-")
    assert data["status"] == "draft"
```

TC-07-P1-API-04: Receive PO Updates Stock

```
def test_receive_po_updates_stock(client, auth_headers, submitted_po,
seeded_product):
    initial_stock = client.get(f"/api/v1/products/{seeded_product['id']}",
                                headers=auth_headers).json().get("stock_level", {})
    response = client.patch(f"/api/v1/orders/{submitted_po}/receive",
headers=auth_headers)
    assert response.status_code == 200
    updated = client.get(f"/api/v1/products/{seeded_product['id']}",
                                headers=auth_headers).json()
    new_stock = updated.get("stock_level", {})
    assert new_stock.get("quantity_on_hand", 0) >
initial_stock.get("quantity_on_hand", 0)
```

TC-07-P1-API-05: Low Alerts Returns Correct Products

```
def test_low_alerts(client, auth_headers):
    response = client.get("/api/v1/stock/low-alerts", headers=auth_headers)
    assert response.status_code == 200
    assert isinstance(response.json(), list)
```

TC-07-P1-API-06: Supplier Catalog Returns Products

```
def test_supplier_catalog(client, auth_headers, seeded_supplier):
    response = client.get(f"/api/v1/suppliers/{seeded_supplier['id']}/catalog",
                                headers=auth_headers)
```

```
assert response.status_code == 200
assert isinstance(response.json(), list)
```

TC-07-P1-API-07: Filter Products by Category

```
def test_filter_by_category(client, auth_headers):
    response = client.get("/api/v1/products?category=grocery",
        headers=auth_headers)
    assert response.status_code == 200
    for p in response.json():
        assert p["category"] == "grocery"
```

TC-07-P1-API-08: Dashboard Returns Metrics

```
def test_dashboard(client, auth_headers):
    response = client.get("/api/v1/dashboard", headers=auth_headers)
    assert response.status_code == 200
    data = response.json()
    for field in ["total_products", "low_stock_count", "out_of_stock_count",
        "open_po_count", "total_stock_value"]:
        assert field in data
```

DATABASE TESTS (4 cases)

TC-07-P1-DB-01: SKU Unique Constraint

```
def test_sku_unique(db_session):
    from app.models import Product
    import pytest
    p1 = Product(sku="SKU-GR0-9999", name="Test 1", category="grocery",
        unit_price=100.0, cost_price=80.0)
    db_session.add(p1); db_session.commit()
    with pytest.raises(Exception):
        p2 = Product(sku="SKU-GR0-9999", name="Test 2", category="grocery",
            unit_price=100.0, cost_price=80.0)
        db_session.add(p2); db_session.commit()
```

TC-07-P1-DB-02: StockMovement Linked to Product

```
def test_movement_linked(db_session, seeded_product_db):
    from app.models import StockMovement
    m = StockMovement(product_id=seeded_product_db.id, movement_type="receipt",
        quantity=50, recorded_by="Kiran")
```

```
db_session.add(m); db_session.commit()
assert m.id is not None and m.product_id == seeded_product_db.id
```

TC-07-P1-DB-03: PO Number Unique

```
def test_po_unique(db_session, seeded_supplier_db):
    from app.models import PurchaseOrder
    from datetime import date
    import pytest
    po1 = PurchaseOrder(po_number="PO-2026-9999",
        supplier_id=seeded_supplier_db.id,
        order_date=date.today())
    db_session.add(po1); db_session.commit()
    with pytest.raises(Exception):
        po2 = PurchaseOrder(po_number="PO-2026-9999",
            supplier_id=seeded_supplier_db.id,
            order_date=date.today())
        db_session.add(po2); db_session.commit()
```

TC-07-P1-DB-04: StockLevel One-to-One with Product

```
def test_stock_level_one_to_one(db_session, seeded_product_db):
    from app.models import StockLevel
    import pytest
    s1 = StockLevel(product_id=seeded_product_db.id, quantity_on_hand=100)
    db_session.add(s1); db_session.commit()
    with pytest.raises(Exception):
        s2 = StockLevel(product_id=seeded_product_db.id, quantity_on_hand=50)
        db_session.add(s2); db_session.commit()
```